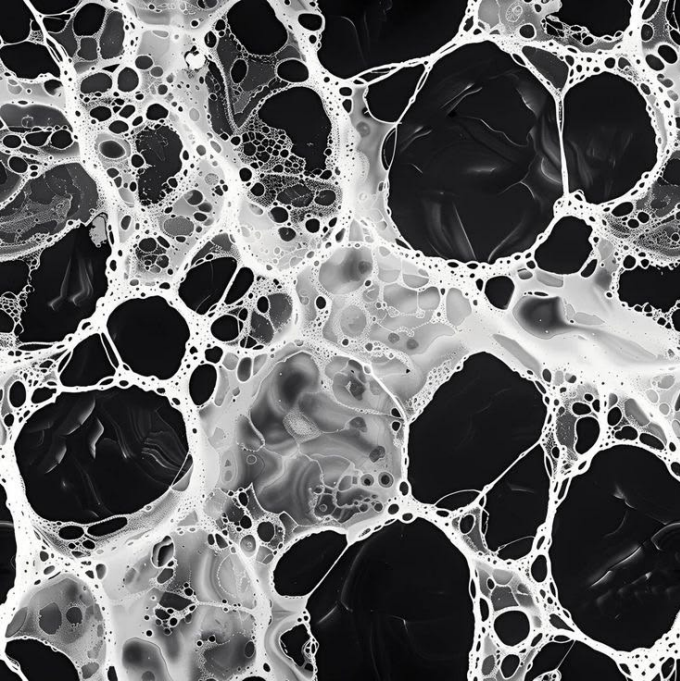




Clasificación de automóviles con redes neuronales

Proyecto 1 – Conjunto de datos
Car Evaluation

Joel Reyes García
Rodrigo Sosa Mendoza

Fundamentos de Inteligencia
Artificial

30 de septiembre de 2026

Perceptrón multicapa 6 → 32 → 16 → 4

1

Planteamiento

Objetivo

- Predecir la **aceptabilidad** de un automóvil a partir de sus características técnicas y de precio.
- Problema de **clasificación multiclase** con 4 categorías.
- Modelo: perceptrón multicapa (MLP) con TensorFlow/Keras.

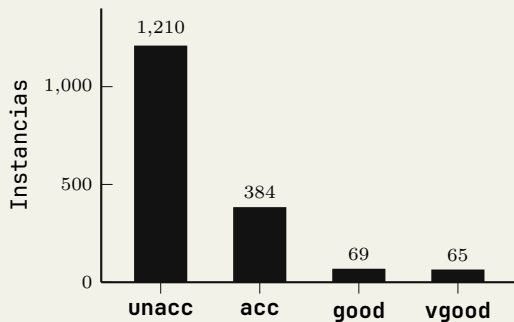


Conjunto de datos

Car Evaluation, repositorio UCI [1]: **1728 instancias**, 6 atributos categóricos, sin valores faltantes.

Atributo	Descripción	Valores
buying	Precio de compra	low, med, high, vhigh
maint	Costo de mantenimiento	low, med, high, vhigh
doors	Número de puertas	2, 3, 4, 5more
persons	Capacidad de personas	2, 4, more
lug_boot	Tamaño de la cajuela	small, med, big
safety	Nivel de seguridad	low, med, high

Distribución de clases



unacc	70.0 %
acc	22.2 %
good	4.0 %
vgood	3.8 %

Conjunto desbalanceado

good y vgood son clases minoritarias.

2

Metodología

Preprocesamiento: atributos

- La red solo recibe números, no texto.
- Cada atributo tiene un **orden natural**: low < med < high < vhigh.
- **OrdinalEncoder** asigna 0, 1, 2, ... respetando ese orden (se le pasa explícito).

Atributos · OrdinalEncoder

```
orden_buying = ["low", "med", "high", "vhigh"]  
# ... un orden por cada atributo  
ordinal_encoder = OrdinalEncoder(categories=[  
    orden_buying, orden_maint, orden_doors,  
    orden_persons, orden_lug_boot, orden_safety])  
X_encoded = ordinal_encoder.fit_transform(X)  
# low->0 med->1 high->2 vhigh->3
```

División de los datos (random_state=42)

Entrenamiento · 1105

Valid. · 277

Prueba · 346

Preprocesamiento: etiqueta

- `LabelEncoder` convierte cada clase en un entero.
- Usa orden alfabético:
`acc=0, good=1, unacc=2, vgood=3.`
- El orden no importa: hay una neurona de salida por clase.
- Queda como entero (no *one-hot*); por eso la pérdida es la versión **sparse**.
- `inverse_transform` regresa la predicción a texto.

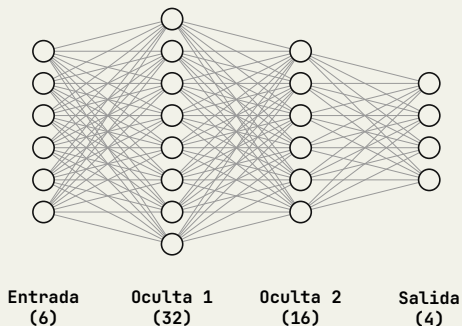
Etiqueta · LabelEncoder

```
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(
    y.values.ravel())

label_encoder.classes_
# ['acc', 'good', 'unacc', 'vgood']
#    0      1      2      3

# Después de predecir: número -> texto
label_encoder.inverse_transform([1])
# ['good']
```

Arquitectura de la red



Capa	Neuronas	Parámetros
Dense · ReLU	32	224
Dense · ReLU	16	528
Dense · Softmax	4	68
Total		820

Funciones de activación y pérdida

ReLU · ocultas

$$f(x) = \text{máx}(0, x)$$

Softmax · salida

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^4 e^{z_j}}$$

Pérdida

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \hat{y}_{i, c_i}$$

- Softmax convierte la salida en probabilidades que suman 1.
- Entropía cruzada categórica *dispersa*: las etiquetas son enteros, no vectores *one-hot*.

Entrenamiento

- Optimizador: Adam
- Épocas: 250
- Tamaño de lote: 16
- Validación: 20 % del entrenamiento
- Métrica: exactitud

proyecto_1.py

```
modelo = Sequential([
    Dense(32, activation="relu", input_shape=(6,)),
    Dense(16, activation="relu"),
    Dense(4, activation="softmax"),
])
modelo.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_train, y_train, epochs=250,
          batch_size=16, validation_split=0.2)
```

3

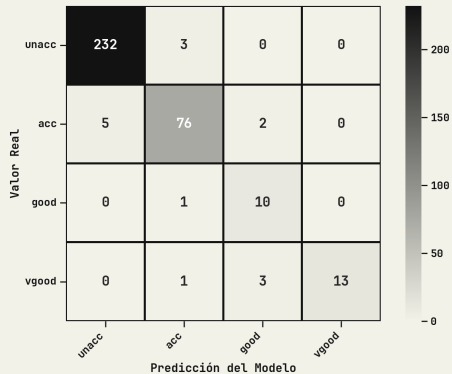
Resultados

Desempeño en el conjunto de prueba

95.66 %

exactitud sobre
346 instancias
no vistas en el
entrenamiento

Matriz de Confusión: Real vs Predicción



Interpretación de la matriz

- Filas: clase real.
Columnas: predicción.
- Diagonal: aciertos,
331 de 346.
- Fuera de la diagonal: 15 errores.
- 14 de 15 errores son entre clases vecinas (unacc↔acc, acc↔good, good↔vgood).
- Nunca confunde unacc con good ni vgood.

Clase	Precisión	Recall	Casos
unacc	0.98	0.99	235
acc	0.94	0.92	83
good	0.67	0.91	11
vgood	1.00	0.76	17

Precisión: de lo que predijo como X , cuánto era X .

Recall: de los X reales, cuántos encontró.

4

Demostración

Guardar el modelo

- Guardar la red evita **reentrenar** cada vez que se usa.
- Se guarda al final del script, después de evaluar.
- **modelo.keras** es un .zip de 35KB con la arquitectura, los **820 pesos** y el estado de Adam.
- **No** guarda los encoders: quien lo use debe codificar con el mismo orden.

codigo/proyecto_1.py

```
modelo.fit(X_train, y_train, epochs=250, ...)  
modelo.evaluate(X_test, y_test)  
modelo.save("app/backend/modelo/modelo.keras")
```

Contenido de modelo.keras

config.json	arquitectura 6→32→16→4, activaciones, pérdida y optimizador
model.weights.h5	los 820 pesos y sesgos + estado de Adam (17500 pasos)
metadata.json	versión de Keras y fecha

Cargar y usar el modelo

- `load_model` reconstruye la red con los mismos pesos.
- Ya no se entrena: solo la **pasada hacia adelante** (inferencia).
- Es lo que hace el backend en cada petición.

proyecto_1.py → modelo.keras → main.py

Uso del modelo guardado

```
modelo = tf.keras.models.load_model("modelo.keras")

# Auto ya codificado (ver Preprocesamiento)
x = np.array([[0, 1, 3, 2, 1, 1]])

p = modelo(x)[0]
# [0.001, 0.999, 0.000, 0.000]
#  acc  good  unacc  vgood

CLASES[np.argmax(p)]
# 'good'
```

Backend con FastAPI

- Carga `modelo.keras` una vez al arrancar.
- Codifica el auto igual que en el entrenamiento.
- `argmax` de las 4 probabilidades da la clase.
- ~10 ms por predicción; ~400/s con 4 procesos.
- En **Docker**, solo accesible desde la red interna.

app/backend/main.py (simplificado)

```
modelo = load_model("modelo/modelo.keras")
app = FastAPI()

@app.post("/api/predecir")
def predecir(auto: Auto):
    entrada = [ORDEN[a].index(getattr(auto, a))
               for a in ORDEN]
    salida = modelo(np.array([entrada]))[0]
    return {
        "clase": CLASES[np.argmax(salida)],
        "probabilidades": dict(zip(CLASES, salida)),
    }
```

Pruébalo



Escanea y
clasifica un auto

fia.jowlab.com

Elige las características y la red
responde en tiempo real con la clase y
sus probabilidades.

5

Conclusiones

Conclusiones

- Una red pequeña (**820 parámetros**) basta para aprender las reglas que definen la aceptabilidad de un automóvil.
- La codificación ordinal preserva el orden de los atributos y simplifica la entrada de la red.
- Los errores se concentran entre clases vecinas: la red aprendió el orden de las categorías. good y vgood son las más débiles (pocos ejemplos por el desbalance).
- Con 250 épocas la exactitud de entrenamiento llega a 100 % y la de validación a 97 %: hay que vigilar el sobreajuste.
- **Trabajo futuro:** pesos por clase, *early stopping* y validación cruzada.

Referencias

- [1] Marko Bohanec. *Car Evaluation*. UCI Machine Learning Repository. 1988. DOI: 10.24432/C5JP48.

Gracias

¿Preguntas?